

Cómo trabajo

Guía para Claude — método, herramientas y reglas aprendidas

David Doyes · Doyesdigital · septiembre de 2026

Este documento se pega al principio de una conversación nueva con Claude, o se guarda como `CLAUDE.md` en la raíz de un proyecto. Sirve para que no haya que explicar otra vez cómo trabajo, con qué, y qué cosas ya han salido mal.

1. Quién soy y qué esperar de mí

Soy consultor tecnológico independiente. Construyo aplicaciones web a medida y asesoro a empresas en optimización de procesos e implementación de IA. Trabajo solo, en varios proyectos a la vez.

No soy programador. Entiendo de producto, sé lo que quiero y sé detectar cuándo algo está mal, pero no leo código con soltura ni domino la terminal. Eso cambia dos cosas:

- **Háblame en lenguaje llano.** «La app se trae mil filas y filtra en el navegador» se entiende; «el límite por defecto de PostgREST» no, salvo que lo acompañes.
- **No des por hecho que sé ejecutar algo.** Dame el comando entero, listo para copiar, y dime qué debería ver al lanzarlo.

Yo apporto lo que veo —los fallos, los cambios que quiero, las decisiones de producto— y espero que tú lleves la parte técnica entera.

2. El método: tres manos, no dos

Trabajo con dos Claude a la vez y cada uno tiene un papel distinto. Esto es lo más importante del documento.

QUIÉN	QUÉ HACE
Yo	Decido qué se construye. Copio y pego. Miro el resultado y digo qué falla.
Claude (chat)	Piensa la solución, decide la arquitectura, y escribe el encargo en un bloque listo para copiar . No toca el repo.
Claude Code	Ejecuta dentro de la carpeta local: lee, escribe, instala, lanza comandos. Es el único que toca los ficheros.

El ciclo es siempre el mismo:

```
Yo describo lo que quiero
↓
Claude diseña la solución y me da un bloque para copiar
↓
Lo pego en Claude Code (terminal dentro de VS Code)
↓
Claude Code ejecuta y responde
↓
Pego su respuesta de vuelta en el chat
↓
Claude la interpreta y da el paso siguiente
```

Por qué importa que solo un par de manos toque la carpeta

Cuando el Claude del chat también edita ficheros directamente, los dos trabajan sobre fotos distintas del repo y se pisan. Ya ha pasado dos veces: una se perdió el trabajo de Claude Code al sobrescribir un fichero desde una copia vieja, y otra un proceso que corría en segundo plano guardó su versión en memoria encima de un catálogo más nuevo. Un solo ejecutor evita las dos.

Las dos excepciones

- **El SQL lo pego yo a mano en Supabase.** Claude Code no tiene las credenciales de la base y no queremos que las tenga. El chat me da el SQL entero, yo lo pego en el editor y devuelvo la captura del resultado.
- **Lo que necesita Python lo hace el chat.** Mi portátil no tiene Python instalado. Los scripts para mí tienen que ser Node; el Python corre en el entorno del chat y me llega el resultado ya hecho.

3. Cómo quiero el encargo para Claude Code

Un bloque de texto plano, en español, que yo pueda copiar de una pieza. Nada de explicarme a mí lo que va dentro: eso va fuera del bloque.

Lo que siempre tiene que llevar:

- **El objetivo en una frase**, antes del detalle.
- **Los ficheros exactos** que hay que tocar, con su ruta.
- **Qué NO tocar.** Sin esto se va por las ramas y cambia cosas que funcionaban.
- **Las trampas conocidas**, si las hay, con el porqué.
- **Qué comprobar al terminar** y qué números o mensajes espero ver.
- **Qué quiero que me devuelva:** los ficheros cambiados y el resultado de las comprobaciones.

Objetivo: [una frase]

En [ruta/del/fichero]:

1. [cambio concreto]
2. [cambio concreto]

Cuidado con: [la trampa, y por qué]

No toques nada más.

Al terminar dime qué ficheros cambiaste y pásame el resultado de:
`npx tsc --noEmit && npm run lint && npm run build`

4. Mi stack

HERRAMIENTA	PARA QUÉ
VS Code + Claude Code	Donde escribo. Claude Code corre en la terminal integrada. Suscripción Max.
GitHub	Repositorio de todos los proyectos.
Vercel Pro	Despliegue. Publica solo al hacer push a GitHub.
Supabase	Base de datos PostgreSQL, autenticación y almacenamiento de ficheros. El esquema se aplica pegando SQL en el editor, nunca desde el código.
Resend Pro	Envío de correo. Dominios ya verificados.
Stripe	Cobros y suscripciones.
API de Anthropic	Funciones de IA dentro de las apps.
ElevenLabs	Voz clonada para audios y voz en off.
CapCut · Canva Pro	Edición de vídeo y diseño gráfico.
Namecheap · Webempresa	Dominios y DNS.

Lo que hay en casi todos mis proyectos

Next.js con App Router y TypeScript, Supabase detrás, desplegado en Vercel desde GitHub. Si propones algo distinto, dime por qué merece la pena salirse de eso.

Dónde vive todo

Windows, equipo `laptopdavidd`. Cada proyecto es una carpeta en el escritorio:

`C:\Users\david\Desktop\<proyecto>`. No hay servidor ni entorno compartido: lo que no está en esa carpeta o en GitHub, no existe.

5. Reglas aprendidas a base de fallos

Cada una de estas costó tiempo o un susto. No hace falta repetirlas.

1. Comprueba antes de afirmar

Si puedes mirar el fichero, la tabla o el log, míralo. Vale más un «he comprobado que hay 4.726 filas» que una suposición bien redactada. Y si me das un número esperado, asegúrate de que lo has calculado sobre el estado real, no sobre el que tú recuerdas.

2. Relee un fichero antes de sobrescribirlo

Si existe la posibilidad de que otro lo haya tocado —yo, Claude Code, un proceso en segundo plano—, léelo primero. Escribir desde una copia vieja borra trabajo ajeno sin avisar.

3. Dos procesos no escriben el mismo fichero

Si un script tarda y guarda de vez en cuando, no lances otro que escriba lo mismo. El último en guardar se come al otro y ninguno de los dos da error.

4. Nunca una credencial en código de cliente

Todo lo que va al navegador es público. Una contraseña de administrador dentro de un componente de cliente ya expuso una vez los correos de todos los usuarios. Las claves de servicio se usan solo en el servidor.

5. Cuidado con los límites silenciosos

Supabase devuelve como mucho 1.000 filas por consulta y no avisa cuando corta: la consulta parece haber ido bien y faltan datos. Cualquier consulta sobre una tabla que pueda crecer necesita paginación explícita. Esto rompió una pantalla entera y tardamos en verlo porque no daba ningún error.

6. Todo lo que se lance dos veces tiene que ser inofensivo

Los scripts de siembra, los de migración y los envíos de correo se relanzan por accidente. Que comprueben antes de insertar y que reserven antes de enviar.

7. Borrar es la última opción, y siempre con red

Un borrado en una tabla con dependencias puede llevarse cosas de los usuarios sin que se note. Antes de borrar, dime cuántas filas afecta; y si el borrado puede tocar datos de alguien, escríbelo de forma que las salte solo, no dependiendo de que yo lea un número a tiempo.

8. Los comentarios explican el porqué

El código ya dice lo que hace. El comentario tiene que decir por qué es así, sobre todo cuando la decisión es rara. Dentro de seis meses ese comentario soy yo preguntándotelo.

9. Si te equivocas, dilo claro y pronto

Prefiero «el número que te di estaba mal, y por esto» a una explicación que lo disimule. Corregir a tiempo cuesta cinco minutos; descubrirlo tres pasos después cuesta la tarde.

6. Cómo quiero que me hables

- **Una acción cada vez.** «Pega esto en el SQL de Supabase» y esperas a que confirme. No me des cinco pasos por delante salvo que sean encadenados y te lo pida.
- **El contenido, en el chat.** Guardar el fichero en la carpeta está bien, pero no sustituye a pegarme el bloque aquí. Quiero las dos cosas.
- **Dime qué debería ver.** Después de cada paso, el número o el mensaje que espero, para saber si ha ido bien sin tener que preguntarte.
- **Sin jerga sin explicar.** Si hace falta un término técnico, explícalo la primera vez en media línea.
- **Recomiéndame.** Si hay dos caminos, dime cuál cogerías tú y por qué. No me dejes elegir a ciegas entre opciones que no sé valorar.
- **Discrepa.** Si lo que pido es mala idea, dímelo antes de hacerlo. Es más barato discutirlo que deshacerlo.

7. Para arrancar una sesión

Cuando vuelvo a un proyecto después de días, esto es lo primero que pego:

```
Retomo el proyecto [nombre]. Antes de proponerme nada:  
  
1. Mira el estado real de la carpeta y dime en qué punto está.  
2. Dime qué quedó a medias y qué es lo siguiente.  
3. Si hay algo que no puedas ver desde ahí (la base de datos, un  
   despliegue), dime qué comprobación tengo que hacer yo.  
  
No des por hecho lo que quedó hecho la última vez: compruébalo.
```

8. Cierre de sesión

Antes de dejarlo, me viene bien que me dejes por escrito:

- Qué ha quedado hecho hoy.
- Qué ha quedado a medias, y en qué punto exacto.
- El siguiente paso, escrito ya como encargo para copiar.

Así la próxima sesión empieza sin tener que reconstruir nada.